

SUPERCOLLIDER PVC SCRIPTS

The Supercollider (SC) PVC scripts are designed to give PVC users access to the familiar UNIX shell script layout for setting flag variables plus the power of the SC programming language to do batch processing and data organization. The following offers instruction in how to perform batch processing. First, however, it is necessary to revisit the basics as translated into the SC environment.

Using PVC SC Scripts:

Each of the familiar PVC shell scripts have been translated into a SC function using a translator program also written in SC. While these SC functions should be sufficient to run PVC from SC, the translation program is included with the scripts for anyone interested in translating their own shell scripts.

The translator program was used to create a corresponding SC file for each PVC shell script; inside each script file is a function with the same name, along with two additional functions: *addCommand* and *runCommands*. These functions work together to allow you to build a string of UNIX commands and then run them using the SC method for ASCII strings called *runInTerminal*. A quick look at the bottom of the file named *SC.plainpv* shows how this works.

```
plainpv.value( iterations: 1 ) ;
```

```
runCommands.value ;
```

In the above, the *plainpv* function call adds commands to a string named *commands*, which is eventually run in a terminal window with *runCommands*; *runCommands* opens a terminal window in which processing of all commands listed in the *commands* string can be seen and followed, just as before; commands are processed sequentially in the order they were entered. The commands, as ordered, are posted to the Supercollider post window.

Setting Variables:

You set flag variables inside the named script function using a variable-setting function declared and used inside it called *setRoutineVariables*; this function has all the familiar look and feel of the original shell scripts. Below is an excerpt from the variable-setting function in *plainpv*:

```
//.....  
//*****  
//..... PLAINPV .....  
//*****  
  
//***** OUTPUT *****
```

```

output_file = "~/output.au" ;

//.....

output_data_format = 1 ;
//( 0: Same as input file )
//( 1: integers )
//( 2: rescaled floats )

```

The text layout of variables and comments is identical to the shell scripts, save the SC syntax requirements, which require full decimal numbers (i.e. 1.1, 3.0, etc.), placement of semi-colons at the ends of lines, and quote marks around ASCII file names, as seen in the above output file: "~/output.au"; comments are designated not by number signs (#) but by double slashes or the old */* comment */* notation. Please note, as well, that ~/ may be used for designating the home directory; *runCommands* retrieves your HOME path and substitutes it for all occurrences of "~/" before running the list of commands.

Adding Gen Functions and Other File-making UNIX Commands:

Just as before, you can add UNIX commands--*gen* functions, *reshape*, etc--in and around your setting of variables. However, to do this you must put your command in a string and add it to the developing list of commands using *addCommand*. Below is a typical example of how this is done:

```

pitch_transposition_in_semitones = "/tmp/pitchChange" ;

addCommand.value(
    "gen4 -L1000 0 0 -2 1 2 > /tmp/pitchChange "
);

```

Commands are added in the order in which they are encountered. The creation or change of data files used by scripts can also be made with *addCommand*, as seen in the following file created for *harmonizer*:

```

addCommand.value(
    "echo

    0      500 22050    -0
    4      500 22050    -0

    > ~/harm_data_file ; "
);

```

After variables are set and support commands added, a command designed to run the routine is constructed out of the variable settings and added to the growing list of

commands. Once the function is exited, the accumulated commands can be run using the function *runCommands*.

Batch Processing with the Iterations Argument:

So far, all of this simply duplicates what the UNIX scripts already offer. However, if we change the *iterations* argument in the function call and add in special functions in place of variable-settings, we open the work environment of these scripts to multiple-run, batch processing of the PVC routines. How does this work?

The process begins with the *iterations* argument, as found in:

```
plainpv.value( iterations: 1 ) ;
```

If we change the iterations argument to 2, as in:

```
plainpv.value( iterations: 2 ) ;
```

the *plainpv* function does everything twice; that is, within the function, it sets variables, adds supporting commands, and makes the PVC command twice, resulting in a duplicate run of the *plainpv* settings. This is not very useful, of course. However, if we take the next step and insert valued, iterated functions into any of our variables, we end up with multiple runs that change in accord with iterating variables.

The most important variable to iterate in this manner is, of course, the output file since we naturally want to save and hear the output of each run--otherwise, we simply write each run into the same file, overwriting the previous output! The following is one way to do this for two explicitly named files.

```
output_file = { [ "~/output0.au", "~/output1.au" ].at( iteration ) }.value ;
```

We could simplify and automate the naming and creation of files by using the following, which constructs multiple, numbered files using the iteration variable.

```
output_file = { "~/output" ++ iteration ++ ".au" }.value ;
```

Other variables can be similarly iterated; for example, different pitch transpositions might be specified with an array of semi-tone transpositions.

```
pitch_transposition_in_semitones = {  
    [ -1, 2, 4 ].at( iteration )  
}.value ;
```

It is even possible to do things like the following, which builds a different *gen4* UNIX command for rewriting the *pitchChange* file with each iterated pass.

```
pitch_transposition_in_semitones = "/tmp/pitchChange" ;
```

```
{  
    c = "gen4 -L1000 0 0 -2 1 " ++  
        { [1, 2, 3].at( iteration ) }.value ++  
        " > /tmp/pitchChange " ;  
    addCommand.value( c ) ;  
}.value ;
```

Interestingly, a unique file name for the pitch function variable is not needed in the structure above since the appropriate command is run just before its values are needed. Of course, one could make a series of control functions beforehand, and access them as desired, as in the following.

```
pitch_transposition_in_semitones = {  
    [ "/tmp/pitchChange0", "/tmp/pitchChange1", "/tmp/pitchChange2" ].at( iteration )  
}.value ;
```

This type of approach offers the opportunity to use predetermined function segments, organized in sequences, to drive a series of PVC runs. For example, a pitch and amplitude tracked bird song, human voice, or melodic instrument sequence could be analyzed and broken into sequences for use in a re-synthesis scheme using different source files to re-synthesize different segments, all, of course, with the assistance of automated, batch processing. Imagine the possibilities.

The above should get you started. More later.....